

手机相机内参标定实验报告

课程作业标题：待填写

姓名：待填写 学号：待填写
课程/班级：待填写

数据状态声明：已使用 18 张真实手机 JPEG 完成一次 OpenCV 标定；具体摄像头模式和方格物理边长仍待确认。

1 实验目的

本实验面向手机相机，使用 Python 调用 OpenCV 完成基于平面棋盘格的相机标定。实验目的如下：

1. 理解世界坐标系、相机坐标系、图像坐标系和像素坐标系之间的关系，掌握针孔相机模型；
2. 区分相机内参数与外参数，理解主点、焦距和镜头畸变系数的物理含义；
3. 掌握棋盘格角点检测、亚像素优化和 `cv2.calibrateCamera` 的基本使用方法；
4. 输出相机内参矩阵、畸变系数和每幅有效图像对应的位姿，并用重投影误差评价标定结果；
5. 在真实采集数据落地后，分析标定板覆盖范围、拍摄姿态和角点检测质量对结果的影响。

本报告只记录实验方法和真实结果的登记位置。手机型号、图像数据及标定数值必须由实际采集和程序运行产生，不以预设值替代实测结果。

2 实验原理

2.1 坐标系与针孔成像模型

世界坐标系用于描述标定板上点的三维位置，相机坐标系以相机光心为原点，图像坐标系通常在成像平面上描述连续坐标，像素坐标系则以图像左上角为参考描述离散像素位置。设世界坐标系中的空间点为 $\mathbf{X}_w = (X_w, Y_w, Z_w)^T$ ，相机坐标系中的点为 $\mathbf{X}_c = (X_c, Y_c, Z_c)^T$ 。相机外参数由旋转矩阵 $\mathbf{R}$ 和平移向量 $\mathbf{t}$ 给出：

$$\mathbf{X}_c = \mathbf{R}\mathbf{X}_w + \mathbf{t}. \quad (1)$$

在针孔模型下，令 $x = X_c/Z_c$ 、 $y = Y_c/Z_c$ 为归一化图像坐标，像素坐标与空间点的关系可写为

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} [\mathbf{R} \quad \mathbf{t}] \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}, \quad s \neq 0, \quad (2)$$

其中 (u, v) 是像素坐标， $\mathbf{K}$ 是相机内参矩阵：

$$\mathbf{K} = \begin{bmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}. \quad (3)$$

这里 f_x, f_y 是以像素为单位的水平和竖直焦距， (c_x, c_y) 是主点， γ 是像素轴非正交时的倾斜参数。普通手机图像通常将 γ 视为零，因此程序输出的矩阵一般接近 $\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$ 。

2.2 镜头畸变模型

实际镜头会使理想针孔投影偏离观测像素。令 $r^2 = x^2 + y^2$ ，常用的径向畸变模型为

$$\begin{aligned} x_r &= x (1 + k_1 r^2 + k_2 r^4 + k_3 r^6), \\ y_r &= y (1 + k_1 r^2 + k_2 r^4 + k_3 r^6). \end{aligned} \quad (4)$$

其中 k_1, k_2, k_3 为径向畸变系数。切向畸变由镜头与成像平面未完全平行等因素引起, 可写为

$$\begin{aligned} x_d &= x_r + 2p_1xy + p_2(r^2 + 2x^2), \\ y_d &= y_r + p_1(r^2 + 2y^2) + 2p_2xy. \end{aligned} \quad (5)$$

最终像素坐标由

$$u = f_x x_d + \gamma y_d + c_x, \quad v = f_y y_d + c_y \quad (6)$$

得到。OpenCV 常见的五参数输出顺序为 $(k_1, k_2, p_1, p_2, k_3)$; 若实际接口返回更多参数, 报告应按程序输出的顺序和维数登记。

2.3 平面标定与张正友法思想

将棋盘格所在平面定义为 $Z_w = 0$, 并以方格边长为单位建立标定板坐标。此时平面点到图像点的映射可由单应矩阵 $\mathbf{H}$ 表示:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} X_w \\ Y_w \\ 1 \end{bmatrix}, \quad \mathbf{H} = \mathbf{K} [\mathbf{r}_1 \quad \mathbf{r}_2 \quad \mathbf{t}], \quad (7)$$

其中 $\mathbf{r}_1, \mathbf{r}_2$ 是旋转矩阵的前两列。对同一棋盘格拍摄多个不同姿态的图像, 每幅图像都会产生一个平面单应约束。利用旋转列向量的正交性和单位长度约束, 可以从多幅图像联合估计内参数, 再恢复各幅图像的外参数。最后, 以所有角点的观测像素与模型投影像素之间的差异为目标进行非线性优化, 从而同时细化内参数、畸变参数和外参数。

程序中, 棋盘格的三维点位于 $Z_w = 0$ 平面, 角点检测得到二维像素点; `cv2.calibrateCamera` 根据这些对应点执行上述标定流程。标定板规格和方格实际边长必须与真实物体一致, 否则 $\mathbf{t}$ 的尺度及由此得到的几何量会失去物理意义。

2.4 重投影误差

设第 j 幅图像中第 i 个角点的观测像素为 $\mathbf{q}_{j,i}$, 由估计参数通过 `cv2.projectPoints` 投影得到的像素为 $\hat{\mathbf{q}}_{j,i}$ 。单点重投影误差定义为

$$e_{j,i} = \|\hat{\mathbf{q}}_{j,i} - \mathbf{q}_{j,i}\|_2. \quad (8)$$

若总有效角点数为 M , 则平均重投影误差和全局均方根误差分别为

$$\bar{e} = \frac{1}{M} \sum_{j,i} e_{j,i}, \quad e_{\text{RMS}} = \sqrt{\frac{1}{M} \sum_{j,i} e_{j,i}^2}. \quad (9)$$

对第 j 幅图像, 程序还计算单幅 RMS 及其算术平均:

$$e_{j,\text{RMS}} = \sqrt{\frac{1}{M_j} \sum_i e_{j,i}^2}, \quad \bar{e}_{\text{view}} = \frac{1}{n} \sum_{j=1}^n e_{j,\text{RMS}}, \quad (10)$$

其中 M_j 是第 j 幅图像的角点数, n 是有效图像数。`rms_error` 是 OpenCV 返回的全局 RMS。

`mean_reprojection_error` 是式9中的点级平均欧氏距离。

`mean_per_image_rms` 是式10中的逐图 RMS 算术平均。三者统计口径不同, 不能互相替代。误差越小通常说明模型对已检测角点的拟合越好, 但仍需结合图像覆盖范围、角点质量和独立验证图像综合判断。

3 实验环境

软件环境取自本次工作区可直接检查到的运行环境。Python 解释器使用 `/Users/jiaqiaosu/miniconda3/bin/python`; 该解释器能够导入 `cv2`, 版本由命令 `cv2.__version__` 确认。JPEG 文件的 EXIF 记录设备为 Apple iPhone 15 Pro。具体摄像头模式仍按采集记录保留为待确认, 不由手机型号推断。

表1中的软件版本、JPEG 设备信息、图像尺寸、内角点配置和图像统计已经由本机命令、EXIF 和 `camera_params.npz` 核实。程序本次以 `square_size=1.0` 运行, 表示每格一个相对单位, 不代表米、厘米或毫米的实测边长。统一缩放不改变以像素表示的相机内参和畸变系数, 但会改变平移向量的尺度; 因此平移向量的物理单位仍待方格边长实测后确定。

表 1: 任务 2 实验环境与采集记录

类别	项目	记录值
操作系统	系统与架构	macOS 26.6.2, Darwin 25.6.0, arm64
软件	Python	3.13.12
软件	OpenCV	4.10.0 (cv2)
软件	主要接口	findChessboardCorners、cornerSubPix
软件	主要接口	calibrateCamera、projectPoints
采集硬件	手机型号	Apple iPhone 15 Pro (JPEG EXIF)
采集硬件	使用摄像头/镜头模式	待确认 (前置或后置及具体镜头模式)
图像	图像分辨率	5712×4284
标定板	内角点规格	8×5
标定板	方格物理边长	待实际测量
数据	输入图像总数	18 张 JPEG (task2/images_jpg/)
数据	有效/失败图像数	18 / 0

4 实验数据采集

4.1 标定板与坐标约定

使用具有规则黑白方格的平面棋盘格作为标定板。程序中的 `--checkerboard` 表示每行和每列的内角点数, `--square-size` 表示相邻方格的边长;两项应与实物逐项核对。本次运行使用 8×5 个内角点和 `square_size=1.0` 的相对单位, 物理方格边长尚未实测。将左上角内角点设为 (0,0,0), 沿棋盘格两条边方向分别建立 X_w 和 Y_w 轴, 所有标定板点的 Z_w 坐标取为零。这一约定与式7的平面模型一致。

4.2 拍摄要求

本次输入为 `task2/images_jpg/*.jpg` 下的 18 张 JPEG, 尺寸均为 5712 × 4284, JPEG EXIF 记录设备为 Apple iPhone 15 Pro。原始 `task2/images/` 下的 HEIC 文件不作为本次 OpenCV 直接输入, 因为当前 `cv2.imread` 无法读取它们。前置或后置摄像头及具体镜头模式尚未由采集记录确认。

拍摄时应同时改变棋盘格相对于镜头的距离、平移位置和倾斜姿态, 使其覆盖图像中央、四周及不同深度; 不能只使用若干张几乎正对且姿态相同的照片。棋盘格应尽量完整、清晰, 避免严重反光、运动模糊、遮挡和过曝。现有文件的姿态覆盖记录尚未整理, 需在提交前依据原始照片补充。

程序实际读取 `task2/images_jpg/`, 保留原文件名, 以便将 NPZ 中的 `image_paths` 与每幅图像的检测状态对应。采集和配置记录见表2。

表 2: 手机标定数据登记表

项目	记录内容	当前状态
手机型号	制造商与型号	Apple iPhone 15 Pro (JPEG EXIF)
摄像头	前置或后置及镜头模式	待人工确认
图像分辨率	宽 × 高 (像素)	5712×4284
图像数量	输入 JPEG 总数	18
棋盘格	每行、每列内角点数	8×5
方格尺寸	实际边长及单位	待实测; 本次运行每格相对单位为 1.0
拍摄姿态	位置、距离、倾角覆盖情况	待根据采集记录补充

本次标定数值来自 18 张真实 JPEG 的实际运行; 尚未确认的字段仅限于具体摄像头/镜头模式、方格物理边长和姿态覆盖记录。由于 `square_size=1.0` 只是统一相对尺度, 内参和畸变的像素结果可以直接报告, 但平移向量不能解释为米制长度。

5 实验步骤

1. 准备平面棋盘格, 实际测量方格边长, 并记录每行、每列内角点数。
2. 使用手机在不同距离、平移位置和倾角下拍摄多幅清晰照片, 记录手机型号、摄像头模式和图像分辨率。
3. 将照片放入输入目录, 逐张读取并检查图像是否成功加载; 读取失败的文件只记录为失败项, 不应使其余图像停止处理。

4. 使用 `cv2.cvtColor` 将彩色图像转换为灰度图，再使用 `cv2.findChessboardCorners` 检测棋盘格内角点。检测成功的图像记录二维角点，检测失败的图像输出文件名和失败状态。
5. 对检测成功的角点调用 `cv2.cornerSubPix` 进行亚像素优化，并将优化终止条件和窗口设置保持为程序中的明确配置。
6. 根据标定板内角点规格和方格边长生成 $Z_w = 0$ 的三维物点，将其与每幅图像的二维角点一一对应。点的几何定义遵循式7。
7. 检查有效图像数量和图像尺寸一致性。若有效图像数量不足以稳定估计参数，则给出提示并要求补充采集；不同尺寸的图像不能直接混合标定。
8. 调用 `cv2.calibrateCamera` 估计相机内参矩阵、畸变系数以及各幅图像的旋转向量和平移向量。内参矩阵的含义见式3，畸变模型见式4和式5。
9. 对每幅有效图像使用 `cv2.projectPoints` 重投影三维物点，依据式8计算角点误差，并汇总平均重投影误差和 RMS。
10. 将相机矩阵、畸变系数、RMS、平均重投影误差、有效图像数及每张图像的检测状态保存到输出结果中；如程序启用校正功能，再使用 `cv2.undistort` 生成校正图像。
11. 将实际运行日志中的结果填入第7节，并按照第8节分析参数合理性和数据质量。

6 程序设计

6.1 输入、配置与输出

程序以带 `argparse` 的 Python 脚本作为入口，从 `task2/images_jpg/` 读取 JPEG。--checkerboard 设置内角点列数和行数，--square-size 设置每格边长数值。--images 设置目录、单张图片或 glob，--output-dir 设置输出目录。--min-images 设定最少有效视图数。

本次真实运行的配置为 8×5 、`square_size=1.0` 和 `--min-images 3`。

对每张输入图像，程序依次执行读取、灰度化、角点检测和亚像素优化。成功样本加入标定点列表，并输出 [OK] 文件名；读取失败、角点检测失败或分辨率不一致的样本输出 [FAILED] 文件名及原因，并汇总失败计数。当前运行的 18 张 JPEG 全部成功，因此读取失败、角点失败和尺寸不一致均为 0。`task2/images/` 中的 HEIC 文件虽会被识别为候选扩展名，但当前 `cv2.imread` 无法直接读取，不能与 JPEG 混用。程序在标定前拒绝有效图像不足和混合分辨率，并在需要时自动创建输出目录。

6.2 核心计算

三维物点由棋盘格坐标生成，二维像点来自优化后的角点。调用 `cv2.calibrateCamera` 后，程序获得内参矩阵 $\mathbf{K}$ 、畸变系数以及每幅有效图像的旋转向量和平移向量。调用时传入 `objectPoints`、`imagePoints` 和 `imageSize`；其投影关系对应式2，畸变参数的登记顺序与 OpenCV 实际返回结果一致。

误差计算使用 `cv2.projectPoints` 将每幅图像的三维物点重投影到像素平面，然后与检测到的二维角点逐点相减。程序同时保存 OpenCV 返回的 `rms_error`、逐图 RMS 数组 `per_image_reprojection_error`、点级全局平均 `mean_reprojection_error` 和逐图 RMS 算术平均 `mean_per_image_rms`。三者的统计定义见式9和式10，不能将一个数值重复标为多个指标。

6.3 可复制的运行命令

本机已验证以下解释器可导入 OpenCV 4.10.0。代码清单1中的第一条命令用于查看完整参数，第二条命令复现本次真实标定；第三条命令在隔离临时目录生成角点和去畸变图件。

Listing 1: 任务 2 程序命令

```
/Users/jiaqiaosu/miniconda3/bin/python task2/calibrate_camera.py --help
/Users/jiaqiaosu/miniconda3/bin/python task2/calibrate_camera.py \
  --checkerboard 8x5 --square-size 1.0 \
  --images "task2/images_jpg/*.jpg" \
  --output-dir task2/output --min-images 3
/Users/jiaqiaosu/miniconda3/bin/python task2/calibrate_camera.py \
  --checkerboard 8x5 --square-size 1.0 \
  --images "task2/images_jpg/*.jpg" \
  --output-dir /private/tmp/task2-final-visual-run --min-images 3 \
  --save-corners --undistort
```

本次第三条命令产生的临时 `camera_params.npz` 仅用于复核和生成图件，不作为报告结果来源。`camera_params.npz` 中的 21 个字段与程序一致：

- 参数字段：`camera_matrix`、`distortion_coefficients`、`rotation_vectors`、`translation_vectors`；
- 误差字段：`rms_error`、`per_image_reprojection_error`、`mean_reprojection_error`、`mean_per_image_rms`；
- 配置字段：`image_size`、`checkerboard`、`square_size`；
- 路径与统计字段：`image_paths`、`valid_image_paths`、`failed_image_paths`、`failure_reasons`、`total_image_count`、`valid_image_count`、`failed_image_count`、`read_failure_count`、`corner_detection_failure_count`、`size_mismatch_count`。

6.4 结果持久化与可复核性

最终参数写入 `task2/output/camera_params.npz`，并在标准输出中打印矩阵、畸变系数、有效图像数、三类误差和内参分量。保存文件同时记录图像尺寸、棋盘格配置、平移/旋转向量、候选/有效/失败路径及失败原因，使表3中的数字可由唯一 NPZ 回读核对。程序不负责推断手机模式或方格物理边长；这些信息必须来自采集记录。

7 实验结果

本次结果直接回读自 `task2/output/camera_params.npz`，对应 `task2/images_jpg/*.jpg` 的 18 张真实 JPEG。程序实际配置为 `checkerboard=8x5`、`square_size=1.0`（每格相对单位）和 `min_images=3`。表3中的数值不是估计或手抄占位。

$$\mathbf{K} = \begin{bmatrix} 4082.15290893 & 0 & 2883.66417823 \\ 0 & 4083.58657686 & 2107.20293734 \\ 0 & 0 & 1 \end{bmatrix}. \quad (11)$$

$$\mathbf{d} = [0.20245123, -0.29420837, -0.00326679, 0.00308671, -1.04554325], \quad (12)$$

其中 $\mathbf{d}$ 按 OpenCV 的常见顺序表示 $(k_1, k_2, p_1, p_2, k_3)$ 。

表 3: 手机相机标定结果（由唯一结果文件回读）

类别	指标	实际值
数据	输入路径	<code>task2/images_jpg/*.jpg</code>
数据	候选图像总数	18
数据	有效标定图像数	18
数据	失败图像数	0（读取、角点、尺寸均为 0）
数据	图像尺寸（宽 × 高）	5712×4284
配置	棋盘格内角点	8×5
配置	<code>square_size</code>	1.0（每格相对单位）
内参	相机矩阵 $\mathbf{K}$	见式11
内参	f_x	4082.15290893
内参	f_y	4083.58657686
内参	c_x	2883.66417823
内参	c_y	2107.20293734
畸变	$(k_1, k_2, p_1, p_2, k_3)$	见式12
误差	OpenCV RMS（像素）	2.06424401
误差	逐角点欧氏距离全局平均（像素）	1.65696931
误差	逐图 RMS 算术平均（像素）	1.90750642
外参	旋转/平移向量形状	18×3×1；平移尺度未定

OpenCV RMS 是 `cv2.calibrateCamera` 返回的全局均方根误差。

逐角点欧氏距离全局平均是 `mean_reprojection_error`，对全部 $18 \times 40 = 720$ 个有效角点求平均。逐图 RMS 算术平均是 `mean_per_image_rms`，对 18 个单幅 RMS 求算术平均。

三者定义不同，不能把 1.65696931、1.90750642 和 2.06424401 互相替代。逐图误差数组保存在 `per_image_reprojection_error` 字段中；旋转向量和平移向量分别保存在 `rotation_vectors` 和 `translation_vectors` 字段中。

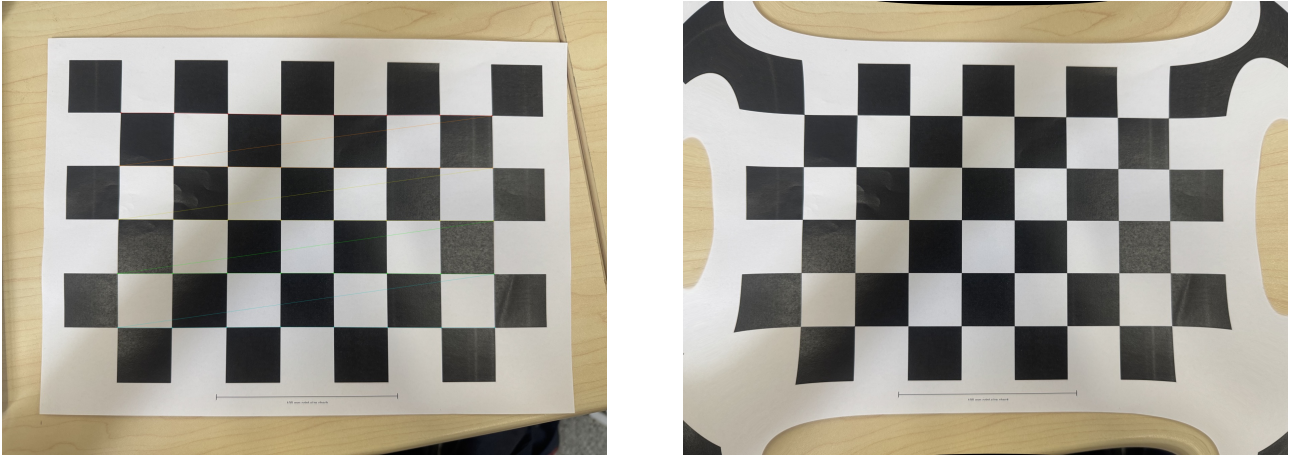


图 1: 代表性图像 IMG_5486.jpg 的结果: 左为程序生成的角点检测可视化, 右为使用同一组标定参数生成的去畸变图像。图件来自临时可视化运行, 参数数值仍以 task2/output/camera_params.npz 为准。

图1只展示 18 张有效图像中的 1 张, 避免将重复图件全部塞入报告。左图用于核对角点检测是否覆盖 8×5 个内角点, 右图用于观察畸变校正效果; 两图均由 `--save-corners --undistort` 实际生成。

8 结果分析

本次分析基于 18 张真实 JPEG、唯一结果文件和同一次实际运行日志。结论限定在当前 5712×4284 图像、 8×5 内角点和相对方格尺度的模型范围内, 不将像素误差直接解释为物理测量精度。

8.1 内参合理性

首先检查 f_x 、 f_y 、 c_x 和 c_y 是否为有限的正数, 并将其与图像尺寸和式3逐项核对。由实测内参得到

$$\rho_f = \frac{|f_x - f_y|}{(f_x + f_y)/2} = 0.0003511422 \quad (0.035114\%).$$

该值说明两方向焦距估计很接近, 与当前图像的近似方形像素尺度相容, 但不能据此确认具体镜头模式或排除图像裁剪。图像中心为 $(W/2, H/2) = (2856, 2142)$, 主点偏移向量为

$$(c_x - W/2, c_y - H/2) = (27.66418, -34.79706) \text{ px},$$

其欧氏长度为 44.45382 px, 分别约占图像宽度和高度的 0.7783% 与 1.0377%。因此主点处于图像中心附近, 但仍应将该偏移作为实测结果保留, 而不是强行置为中心。

8.2 畸变与误差

按 OpenCV 的实际输出顺序, 畸变向量为

$$(0.20245123, -0.29420837, -0.00326679, 0.00308671, -1.04554325).$$

这说明本次模型中径向和切向修正项均未被设为零。系数的正负和大小不能单独证明校正效果, 图1中的去畸变图应与原图的棋盘格边缘共同观察。

三种误差分别为 OpenCV RMS 2.06424401 px、逐角点欧氏距离全局平均 1.65696931 px 和逐图 RMS 算术平均 1.90750642 px。三者不同是因为前者对所有角点的平方距离统一汇总, 第二个对点距离取算术平均, 第三个先对每幅图像求 RMS 再对 18 幅图像取平均, 不能用数值大小直接判断程序矛盾。逐图 RMS 范围为 0.850436–3.625463 px, 较大的单幅误差应在需要时回看原图和角点覆盖。

对当前分辨率而言, OpenCV RMS 约为图像宽度的 0.03614%、高度的 0.04818%。这说明误差相对于像素尺寸的比例不大, 但大分辨率会使相同的像素误差看起来具有更小的相对比例; 该指标不能替代独立验证, 也不能直接跨分辨率比较。

8.3 数据质量与姿态覆盖

本次 18 张候选 JPEG 全部通过读取、 8×5 角点检测和分辨率检查，失败图像数为 0，且每幅图像均有 40 个角点。因此当前结果在数据完整性和输入一致性上通过了程序检查。现有报告没有独立的拍摄姿态日志，仍不能仅凭“18/18 成功”断言棋盘格已经均匀覆盖图像中央、边缘、不同距离和倾角；提交前应补充姿态记录，并对逐图 RMS 较大的照片进行人工复核。

由于本次 `square_size=1.0` 只定义每格一个相对单位，旋转向量不受统一尺度影响，内参矩阵和以像素表示的畸变结果也不受统一尺度影响；平移向量与方格物理边长成比例，当前不能解释为米、厘米或毫米。

表 4: 基于真实运行结果的分析摘要		
检查项	实测证据	谨慎结论
焦距一致性	$\rho_f = 0.0003511422$	两方向估计接近
主点位置	偏移 44.45382 px	相对中心有小幅偏移
畸变系数	完整五参数向量	需结合校正图像解读
RMS 与平均误差	2.06424401/1.65696931/1.90750642 px	三种口径均已记录
数据完整性	18 候选，18 有效，0 失败	通过程序输入检查
姿态覆盖	独立姿态日志未提供	不能据此宣称覆盖充分
物理尺度	<code>square_size=1.0</code>	平移向量物理单位未定

综合而言，本次数据足以在当前图像坐标模型下给出一组可复核的相机内参和畸变参数；但由于具体镜头模式、方格物理边长和完整姿态覆盖记录尚未确认，不能将该结果夸大为带物理尺度和全场景精度保证的相机模型。

9 实验结论

本实验使用 18 张真实 iPhone 15 Pro JPEG 图像，在 8×5 个棋盘格内角点配置下完成了 OpenCV 手机相机内参标定。程序输出了相机内参矩阵、五个畸变系数、18 组旋转和平移向量，并记录了 OpenCV RMS、逐角点欧氏距离全局平均和逐图 RMS 算术平均。18 张候选图像全部有效，未出现读取失败、角点检测失败或尺寸不一致。

本次运行使用 `square_size=1.0`，表示每格一个相对单位；因此内参矩阵和像素畸变结果可以用于当前图像坐标模型，但平移向量的物理尺度尚未确定。JPEG EXIF 已确认设备为 Apple iPhone 15 Pro，具体前置/后置摄像头或镜头模式仍待人工确认，方格物理边长和完整姿态覆盖记录也应在正式提交前补齐。上述未决信息不影响本次像素级参数回读，但会限制对外参物理长度和跨设备复现的解释。